Full Stack TypeScript/Node.js Developer - Interview Questions & Answers

Role Level: Intermediate to Senior
Focus Areas: TypeScript, React/Vue.js, Node.js, AWS, Serverless, GraphQL, Docker/Kubernetes
Interview Type: Technical + Behavioral

TABLE OF CONTENTS

TypeScript & JavaScript

Q1: Explain TypeScript's Type System and Advanced Types (Generics, Union, Intersection)

Question: "TypeScript adds a type system to JavaScript. Can you explain how generics work, and when would you use union types vs intersection types? Provide code examples."

Model Answer:

TypeScript's type system provides compile-time type checking, catching errors before runtime. Here are the key advanced concepts:

1. Generics - Reusable Type-Safe Components

```typescript
```

```typescript
// Generic function - works with any type
function createArray<T>(length: number, value: T): T[] {
    return Array(length).fill(value);
}

// Usage
const stringArray = createArray<string>(3, "hello"); // ["hello", "hello", "hel
const numberArray = createArray<number>(2, 5);       // [5, 5]

// Generic class
class Stack<T> {
    private items: T[] = [];

    push(element: T): void {
        this.items.push(element);
    }

    pop(): T | undefined {
        return this.items.pop();
    }
}

// Generic with constraints
function merge<T extends object, U extends object>(obj1: T, obj2: U): T & U {
    return { ...obj1, ...obj2 };
}

const result = merge({ a: 1 }, { b: "hello" }); // { a: 1, b: "hello" }
```

2. Union Types - Multiple Possible Types

typescript

```typescript
// A variable can be one of several types
type Status = "success" | "error" | "loading";
let currentStatus: Status = "success"; // ✓ Valid
// currentStatus = "pending"; // ✗ Error: not in union

// Function accepting multiple types
function processValue(value: string | number): void {
    if (typeof value === "string") {
        console.log(value.toUpperCase()); // String method
    } else {
        console.log(value.toFixed(2));   // Number method
    }
}

// Union with null/undefined
type MaybeString = string | null | undefined;
```

3. Intersection Types - Combining Multiple Types

typescript

```typescript
// Intersection combines all properties
interface Printable {
    print(): void;
}

interface Saveable {
    save(): void;
}

type Document = Printable & Saveable;

const doc: Document = {
    print() { console.log("Printing..."); },
    save() { console.log("Saving..."); }
};

// Real-world example: User with admin capabilities
interface User {
    id: number;
    name: string;
}

interface Admin {
    adminLevel: number;
    permissions: string[];
}

type AdminUser = User & Admin;

const adminUser: AdminUser = {
    id: 1,
    name: "John",
    adminLevel: 5,
    permissions: ["read", "write", "delete"]
};
```

Key Differences:

Concept	Use Case	Example
Union (OR)	Value can be one of several types	`string | number`
Intersection (AND)	Object must have all properties	`User & Admin`
Generics	Reusable, type-safe components	`Array<T>`, `Promise<T>`

Why This Matters:

typescript

```typescript
// Without generics (not type-safe)
function getFirstElement(arr: any[]): any {
    return arr[0];
}
const first = getFirstElement([1, 2, 3]); // Type is 'any'

// With generics (type-safe)
function getFirstElement<T>(arr: T[]): T {
    return arr[0];
}
const first = getFirstElement([1, 2, 3]); // Type is 'number' ✓
```

Red Flags:

- ❌ "I don't use unions, I just use `any`"
- ❌ "Generics are too complicated"
- ❌ Confusing union and intersection

Green Flags:

- ✅ Explains practical use cases
- ✅ Shows understanding of type constraints
- ✅ Mentions discriminated unions

Follow-up Questions:

1. "What's a discriminated union and why is it useful?"
2. "How do you handle type narrowing with unions?"
3. "Can you explain conditional types in TypeScript?"

Q2: What are Type Guards and Discriminated Unions? Provide Examples.

Question: "Explain type guards and discriminated unions. Why are they important? Show me a real-world example."

Model Answer:

Type Guards are techniques to narrow down variable types in conditional blocks.

1. typeof Guard

```typescript
function process(value: string | number): void {
    if (typeof value === "string") {
        console.log(value.toUpperCase()); // value is string here
    } else {
        console.log(value.toFixed(2));   // value is number here
    }
}
```

2. instanceof Guard

```typescript
class Car {
    drive() { return "Driving car"; }
}

class Bike {
    ride() { return "Riding bike"; }
}

function operate(vehicle: Car | Bike): void {
    if (vehicle instanceof Car) {
        console.log(vehicle.drive()); // Car methods available
    } else {
        console.log(vehicle.ride());  // Bike methods available
    }
}
```

3. Discriminated Unions (Best Practice)

A discriminated union has a common property (discriminator) to distinguish types:

```typescript
```

```typescript
// Define tagged union with 'type' discriminator
type ApiResponse =
    | { status: "success"; data: unknown }
    | { status: "error"; error: string }
    | { status: "loading" };

function handleResponse(response: ApiResponse): void {
    // TypeScript knows which properties are available based on status
    switch (response.status) {
        case "success":
            console.log(response.data); // ✓ 'data' exists here
            break;
        case "error":
            console.log(response.error); // ✓ 'error' exists here
            break;
        case "loading":
            console.log("Loading..."); // ✓ No data/error
    }
}
```

Real-World Example: API Result Handler

typescript

```typescript
// Discriminated union for API responses
type ApiResult<T> =
    | { kind: "success"; value: T }
    | { kind: "error"; error: Error }
    | { kind: "pending" };

// Usage with discriminator
const result: ApiResult<User> = { kind: "success", value: { id: 1, name: "John"

if (result.kind === "success") {
    console.log(result.value.name); // Type-safe! ✓
} else if (result.kind === "error") {
    console.log(result.error.message);
} else {
    console.log("Still loading...");
}

// Another example: UI Events
type UIEvent =
    | { type: "click"; x: number; y: number }
    | { type: "scroll"; direction: "up" | "down" }
    | { type: "keypress"; key: string };

function handleEvent(event: UIEvent): void {
    switch (event.type) {
        case "click":
            console.log(`Clicked at ${event.x}, ${event.y}`);
            break;
        case "scroll":
            console.log(`Scrolled ${event.direction}`);
            break;
        case "keypress":
            console.log(`Key pressed: ${event.key}`);
    }
}
```

Why Discriminated Unions?

typescript

```
// ❌ Without discriminated union (error-prone)
type Result = {
    success?: boolean;
    data?: unknown;
    error?: string;
};

const result: Result = { success: true, data: "user" };
if (result.success) {
    // TypeScript doesn't know if data exists!
    console.log(result.data); // Might be undefined
}

// ✅ With discriminated union (type-safe)
type Result =
    | { success: true; data: unknown }
    | { success: false; error: string };

const result: Result = { success: true, data: "user" };
if (result.success) {
    console.log(result.data); // TypeScript knows data exists! ✓
}
```

Follow-up Questions:

1. "Why is a discriminated union better than optional properties?"
2. "How do you handle exhaustiveness checking in switch statements?"

Q3: Explain Decorators and Metadata in TypeScript (Advanced)

Question: "What are decorators? How have you used them? Show an example with a class decorator."

Model Answer:

Decorators are special functions that modify class definitions, methods, or properties. They're heavily used in frameworks like NestJS, Angular.

Enable Decorators in tsconfig.json:

```
json
```

```
{
  "compilerOptions": {
    "experimentalDecorators": true,
    "emitDecoratorMetadata": true
  }
}
```

1. Method Decorator - Logging

typescript

```typescript
// Method decorator factory
function Log(target: any, propertyKey: string, descriptor: PropertyDescriptor)
    const originalMethod = descriptor.value;

    descriptor.value = function(...args: any[]) {
        console.log(`Calling ${propertyKey} with args:`, args);
        const result = originalMethod.apply(this, args);
        console.log(`Result:`, result);
        return result;
    };

    return descriptor;
}

class Calculator {
    @Log
    add(a: number, b: number): number {
        return a + b;
    }
}

const calc = new Calculator();
calc.add(2, 3);
// Output:
// Calling add with args: [2, 3]
// Result: 5
```

2. Class Decorator - Validation

typescript

```typescript
// Class decorator
function Validate(target: Function) {
    const original = target;

    const newConstructor: any = function(...args: any[]) {
        console.log(`Creating instance of ${original.name}`);
        return new original(...args);
    };

    newConstructor.prototype = original.prototype;
    return newConstructor;
}

@Validate
class User {
    constructor(public name: string, public email: string) {}
}

const user = new User("John", "john@example.com");
// Output: Creating instance of User
```

3. Property Decorator - Validation

typescript

```typescript
// Property decorator for validation
function MinLength(length: number) {
    return function(target: any, propertyKey: string) {
        let value: string;

        Object.defineProperty(target, propertyKey, {
            get() { return value; },
            set(newValue: string) {
                if (newValue.length < length) {
                    throw new Error(`${propertyKey} must be at least ${length}
                }
                value = newValue;
            }
        });
    };
}

class Person {
    @MinLength(3)
    name: string = "";
}

const person = new Person();
person.name = "Jo"; // Error: name must be at least 3 characters ✓
```

4. Real-World: NestJS Example

typescript

```
// NestJS uses decorators heavily
@Controller('users')
export class UserController {
    @Get(':id')
    @UseGuards(AuthGuard)
    getUser(@Param('id') id: string): User {
        return new User(id, 'John');
    }

    @Post()
    @UseGuards(AuthGuard)
    createUser(@Body() createUserDto: CreateUserDto): User {
        return new User('1', createUserDto.name);
    }
}
```

Why Decorators?

- ✅ Clean, declarative code
- ✅ Separation of concerns
- ✅ Reusable functionality
- ✅ Metadata attachment

Follow-up Questions:

1. "What's the difference between a decorator factory and a decorator?"
2. "How does metadata work with decorators?"
3. "Have you used decorators in NestJS or Angular?"

Q4: Explain Async/Await vs Promises vs Callbacks. When to use each?

Question: "Compare callbacks, promises, and async/await. Show examples of each and explain when you'd use them."

Model Answer:

Three Paradigms for Handling Asynchronous Code:

1. Callbacks (Oldest - Avoid)

```
typescript
```

```typescript
// Callback example
function fetchUser(id: number, callback: (user: any) => void): void {
    setTimeout(() => {
        const user = { id, name: "John" };
        callback(user);
    }, 1000);
}

fetchUser(1, (user) => {
    console.log(user);
});

// ❌ Callback Hell (pyramid of doom)
function getUser(id: number, cb: (user: any) => void) {
    fetchUser(id, (user) => {
        getPosts(user.id, (posts) => {
            getComments(posts[0].id, (comments) => {
                console.log(comments); // Nested callbacks!
            });
        });
    });
}
```

2. Promises (Better)

typescript

```typescript
// Promise example
function fetchUser(id: number): Promise<User> {
    return new Promise((resolve, reject) => {
        setTimeout(() => {
            if (id > 0) {
                resolve({ id, name: "John" });
            } else {
                reject(new Error("Invalid ID"));
            }
        }, 1000);
    });
}

// Chaining with .then()
fetchUser(1)
    .then(user => {
        console.log(user);
        return getPosts(user.id);
    })
    .then(posts => {
        console.log(posts);
        return getComments(posts[0].id);
    })
    .then(comments => console.log(comments))
    .catch(error => console.error(error));

// Parallel with Promise.all()
Promise.all([
    fetchUser(1),
    fetchUser(2),
    fetchUser(3)
])
    .then(users => console.log(users))
    .catch(error => console.error(error));
```

3. Async/Await (Best - Modern)

typescript

```typescript
// Async/await reads like synchronous code
async function getFullUserData(id: number): Promise<void> {
    try {
        const user = await fetchUser(id);
        const posts = await getPosts(user.id);
        const comments = await getComments(posts[0].id);
        console.log(comments);
    } catch (error) {
        console.error(error);
    }
}

// Parallel operations with async/await
async function getAllUsers(): Promise<User[]> {
    const [user1, user2, user3] = await Promise.all([
        fetchUser(1),
        fetchUser(2),
        fetchUser(3)
    ]);
    return [user1, user2, user3];
}

// Sequential vs Parallel
async function sequential(): Promise<void> {
    const user1 = await fetchUser(1); // Wait 1s
    const user2 = await fetchUser(2); // Wait 1s (total: 2s)
    console.log(user1, user2);
}

async function parallel(): Promise<void> {
    const results = await Promise.all([
        fetchUser(1),
        fetchUser(2)
    ]); // Wait 1s (both in parallel)
    console.log(results);
}
```

Comparison:

Feature	Callbacks	Promises	Async/Await
Readability	Poor	Good	Excellent
Error Handling	Messy	`.catch()`	try/catch ✓
Readability	Left-to-right	Chain	Sequential ✓
Parallel Ops	Complex	`Promise.all()`	`Promise.all()`
Debugging	Hard	Better	Best ✓

Real-World Node.js Example:

typescript

```
// Async function - common in Node.js/Express
async function getUserWithPosts(userId: number): Promise<{ user: User; posts: P
    try {
        // Sequential operations
        const user = await db.query('SELECT * FROM users WHERE id = ?', [userId

        if (!user) {
            throw new Error('User not found');
        }

        // Parallel operations
        const [posts, friends] = await Promise.all([
            db.query('SELECT * FROM posts WHERE userId = ?', [userId]),
            db.query('SELECT * FROM friends WHERE userId = ?', [userId])
        ]);

        return { user, posts };

    } catch (error) {
        console.error('Error fetching user:', error);
        throw error;
    }
}

// Using in Express route
app.get('/users/:id', async (req, res) => {
    try {
        const data = await getUserWithPosts(parseInt(req.params.id));
        res.json(data);
    } catch (error) {
        res.status(500).json({ error: error.message });
    }
});
```

When to Use:

typescript

```
// ✅ Use Async/Await (modern standard)
async function doSomething() {
    const result = await fetch(...);
}

// ✅ Use Promise.all() for parallel operations
const results = await Promise.all([fetch1(), fetch2()]);

// ✅ Use try/catch for error handling
try {
    await doSomething();
} catch (e) {
    console.error(e);
}

// ❌ Avoid callbacks (except for event listeners)
// ❌ Avoid mixing promises and async/await unnecessarily
```

Follow-up Questions:

1. "What's the difference between `Promise.all()` and `Promise.allSettled()`?"
2. "How do you handle timeouts with promises?"
3. "What's the difference between `Promise.race()` and `Promise.all()`?"

Q5: What are Higher-Order Functions and Function Composition in TypeScript?

Question: "Explain higher-order functions and function composition. Show a practical example where you'd use them."

Model Answer:

Higher-Order Function (HOF) is a function that:

- Takes functions as arguments, OR
- Returns a function

1. HOF that Takes a Function as Argument

typescript

```typescript
// Higher-order function for filtering
function filterArray<T>(arr: T[], predicate: (item: T) => boolean): T[] {
    return arr.filter(predicate);
}

// Usage
const numbers = [1, 2, 3, 4, 5];
const evens = filterArray(numbers, n => n % 2 === 0); // [2, 4]

// Another example: map with transform
function mapArray<T, U>(arr: T[], transform: (item: T) => U): U[] {
    return arr.map(transform);
}

const doubled = mapArray(numbers, n => n * 2); // [2, 4, 6, 8, 10]
```

2. HOF that Returns a Function (Currying)

typescript

```typescript
// Function that returns a function
function multiply(a: number) {
    return (b: number) => a * b;
}

const double = multiply(2);
console.log(double(5)); // 10

// Practical example: Logger decorator
function createLogger(prefix: string) {
    return (message: string) => {
        console.log(`[${prefix}] ${message}`);
    };
}

const appLogger = createLogger("APP");
const dbLogger = createLogger("DB");

appLogger("Application started");   // [APP] Application started
dbLogger("Connected to database");  // [DB] Connected to database

// Middleware pattern in Express
function authenticate(secret: string) {
    return (req: Request, res: Response, next: NextFunction) => {
        const token = req.headers.authorization;
        if (token === secret) {
            next();
        } else {
            res.status(401).send("Unauthorized");
        }
    };
}

app.use(authenticate("secret-key"));
```

3. Function Composition

Compose multiple functions into a single function:

typescript

```typescript
// Basic composition
function compose<T, U, V>(
    f: (x: U) => V,
    g: (x: T) => U
): (x: T) => V {
    return (x: T) => f(g(x));
}

const add1 = (x: number) => x + 1;
const multiply2 = (x: number) => x * 2;

const composed = compose(multiply2, add1);
console.log(composed(5)); // (5 + 1) * 2 = 12

// Practical example: Data transformation pipeline
const trim = (str: string) => str.trim();
const toLowerCase = (str: string) => str.toLowerCase();
const removeSpaces = (str: string) => str.replace(/\s/g, '');

const composeSafe = <T, U, V>(
    f: (x: U) => V,
    g: (x: T) => U
) => (x: T) => f(g(x));

const processString = composeSafe(
    removeSpaces,
    composeSafe(toLowerCase, trim)
);

console.log(processString("  HELLO WORLD  ")); // "helloworld"
```

4. Real-World: Middleware Composition in Express

typescript

```typescript
// Middleware composition pattern
type Middleware = (req: Request, res: Response, next: NextFunction) => void;

function compose(...middlewares: Middleware[]) {
    return (req: Request, res: Response, next: NextFunction) => {
        let index = -1;

        const dispatch = (i: number): Promise<void> => {
            if (i <= index) return Promise.reject(new Error('next() called mult
            index = i;

            if (i === middlewares.length) {
                next();
                return Promise.resolve();
            }

            try {
                return Promise.resolve(middlewares[i](req, res, () => dispatch(
            } catch (err) {
                return Promise.reject(err);
            }
        };

        return dispatch(0);
    };
}

// Usage
const authMiddleware = (req: Request, res: Response, next: NextFunction) => {
    if (req.headers.authorization) {
        next();
    } else {
        res.status(401).send('Unauthorized');
    }
};

const loggingMiddleware = (req: Request, res: Response, next: NextFunction) =>
    console.log(`${req.method} ${req.path}`);
    next();
};
```

```
const composed = compose(loggingMiddleware, authMiddleware);
app.use(composed);
```

5. Map/Filter/Reduce (Functional Programming)

typescript

```typescript
// Functional approach to data transformation
const users = [
    { id: 1, name: "John", active: true, age: 30 },
    { id: 2, name: "Jane", active: true, age: 25 },
    { id: 3, name: "Bob", active: false, age: 35 }
];

// Map: Transform data
const names = users.map(u => u.name); // ["John", "Jane", "Bob"]

// Filter: Select data
const active = users.filter(u => u.active);

// Reduce: Aggregate data
const totalAge = users.reduce((sum, u) => sum + u.age, 0); // 90

// Chaining operations
const result = users
    .filter(u => u.active)
    .map(u => ({ name: u.name, age: u.age }))
    .reduce((acc, u) => ({ ...acc, [u.name]: u.age }), {});

// Result: { John: 30, Jane: 25 }
```

Why Higher-Order Functions?

- ✅ Code reusability
- ✅ Separation of concerns
- ✅ Functional programming patterns
- ✅ Middleware/plugin patterns

Follow-up Questions:

1. "What's the difference between composition and currying?"
2. "How do you compose async functions?"
3. "Can you explain the pipe operator?"

Q6: Explain Dependency Injection and How You'd Implement It

Question: "What is Dependency Injection? Why is it important? Show a TypeScript example."

Model Answer:

Dependency Injection (DI) is a design pattern where:

- Dependencies are passed to a class/function (not created inside)
- Improves testability, flexibility, and maintainability

1. Without DI (Tightly Coupled - Bad)

typescript

```typescript
// ❌ Logger is tightly coupled
class UserService {
    private logger = new ConsoleLogger();  // Hard to replace

    createUser(name: string) {
        this.logger.log(`Creating user: ${name}`);
        // Create user...
    }
}

class ConsoleLogger {
    log(message: string) {
        console.log(message);
    }
}

// Problem: Can't test with a mock logger
```

2. With DI (Loosely Coupled - Good)

typescript

```typescript
// ✅ Logger is injected
interface ILogger {
    log(message: string): void;
}

class UserService {
    constructor(private logger: ILogger) {} // Dependency injected

    createUser(name: string) {
        this.logger.log(`Creating user: ${name}`);
        // Create user...
    }
}

class ConsoleLogger implements ILogger {
    log(message: string) {
        console.log(message);
    }
}

// Easy to swap with mock for testing
class MockLogger implements ILogger {
    logs: string[] = [];
    log(message: string) {
        this.logs.push(message);
    }
}

// Usage
const logger = new ConsoleLogger();
const service = new UserService(logger);

// Testing
const mockLogger = new MockLogger();
const testService = new UserService(mockLogger);
testService.createUser("John");
console.log(mockLogger.logs); // ["Creating user: John"]
```

3. DI Container Pattern (NestJS Style)

typescript

```typescript
// Simplified DI Container
class DIContainer {
    private services: Map<string, any> = new Map();

    register(name: string, constructor: Function, dependencies: string[] = [])
        this.services.set(name, { constructor, dependencies });
    }

    get<T>(name: string): T {
        const service = this.services.get(name);
        if (!service) throw new Error(`Service ${name} not found`);

        const dependencyInstances = service.dependencies.map((dep: string) =>
            this.get(dep)
        );

        return new service.constructor(...dependencyInstances);
    }
}

// Usage
interface IDatabase {
    query(sql: string): Promise<any>;
}

class Database implements IDatabase {
    async query(sql: string) {
        return { id: 1, name: "John" };
    }
}

interface IUserRepository {
    getUser(id: number): Promise<User>;
}

class UserRepository implements IUserRepository {
    constructor(private db: IDatabase) {}

    async getUser(id: number) {
        return this.db.query(`SELECT * FROM users WHERE id = ${id}`);
    }
}
```

```
class UserService {
    constructor(private repository: IUserRepository) {}

    async getUser(id: number) {
        return this.repository.getUser(id);
    }
}

// Setup DI container
const container = new DIContainer();
container.register('Database', Database);
container.register('UserRepository', UserRepository, ['Database']);
container.register('UserService', UserService, ['UserRepository']);

// Get the service (all dependencies injected automatically)
const userService = container.get<UserService>('UserService');
```

4. NestJS Example (Production Framework)

typescript

```
// NestJS uses decorators for DI
@Injectable()
export class DatabaseService {
    async query(sql: string) {
        // Query database
    }
}

@Injectable()
export class UserRepository {
    constructor(private db: DatabaseService) {} // Injected automatically

    async getUser(id: number) {
        return this.db.query(`SELECT * FROM users WHERE id = ${id}`);
    }
}

@Injectable()
export class UserService {
    constructor(private userRepo: UserRepository) {} // Injected automatically

    async getUser(id: number) {
        return this.userRepo.getUser(id);
    }
}

@Controller('users')
export class UserController {
    constructor(private userService: UserService) {} // Injected automatically

    @Get(':id')
    async getUser(@Param('id') id: string) {
        return this.userService.getUser(parseInt(id));
    }
}
```

Benefits of DI:

Benefit	Explanation
Testability	Easy to inject mocks/stubs
Flexibility	Swap implementations easily
Maintainability	Loose coupling, easier to change
Reusability	Services are independent

Follow-up Questions:

1. "What's the difference between constructor injection and property injection?"
2. "How does NestJS auto-wire dependencies?"
3. "What are circular dependency issues and how do you solve them?"

React/Vue.js Frontend

Q7: Explain React Hooks and the Difference Between useEffect and useLayoutEffect

Question: "Explain React hooks. What's the difference between `useEffect` and `useLayoutEffect`? When would you use each?"

Model Answer:

React Hooks allow functional components to have state and lifecycle features.

1. useState Hook

typescript

```typescript
// useState manages component state
const [count, setCount] = useState<number>(0);
const [name, setName] = useState<string>("");

return (
    <div>
        <p>Count: {count}</p>
        <button onClick={() => setCount(count + 1)}>Increment</button>
        <input value={name} onChange={e => setName(e.target.value)} />
    </div>
);
```

2. useEffect Hook (Runs After Render)

typescript

```typescript
// useEffect runs AFTER React renders the component
useEffect(() => {
    console.log("Component rendered");

    // Cleanup function (optional)
    return () => {
        console.log("Cleanup");
    };
}, []); // Dependency array

// Different dependency arrays:
useEffect(() => { /* runs on every render */ });
useEffect(() => { /* runs once on mount */ }, []);
useEffect(() => { /* runs when count changes */ }, [count]);

// Fetching data
useEffect(() => {
    const fetchData = async () => {
        const response = await fetch('/api/users');
        const data = await response.json();
        setUsers(data);
    };

    fetchData();
}, []); // Only fetch once on mount
```

3. useLayoutEffect Hook (Runs Before Paint)

typescript

```
// useLayoutEffect runs BEFORE the browser paints
// Use for DOM measurements and synchronous updates

useLayoutEffect(() => {
    // This runs synchronously after DOM mutations but before paint
    const height = elementRef.current?.offsetHeight;
    setMeasuredHeight(height);
}, []);

// Real example: Tooltip positioning
useLayoutEffect(() => {
    if (tooltipRef.current && triggerRef.current) {
        const triggerRect = triggerRef.current.getBoundingClientRect();
        const tooltipRect = tooltipRef.current.getBoundingClientRect();

        // Calculate position based on measurements
        const top = triggerRect.bottom + 10;
        const left = triggerRect.left + triggerRect.width / 2 - tooltipRect.wid

        tooltipRef.current.style.top = `${top}px`;
        tooltipRef.current.style.left = `${left}px`;
    }
}, [isVisible]);
```

Comparison:

Feature	useEffect	useLayoutEffect
Timing	After render (async)	Before paint (sync)
DOM Paint	Can see flashing	No flashing
Use Case	Data fetching, subscriptions	DOM measurements
Performance	Better	Blocks paint

typescript

```
// ✅ Use useEffect for
useEffect(() => {
    // Data fetching
    fetch('/api/data');

    // Event listeners
    window.addEventListener('resize', handleResize);

    // Subscriptions
    subscribe();

    return () => {
        window.removeEventListener('resize', handleResize);
        unsubscribe();
    };
}, []);

// ✅ Use useLayoutEffect for
useLayoutEffect(() => {
    // DOM measurements
    const height = element.offsetHeight;

    // Synchronous state updates
    setMeasuredHeight(height);
}, []);
```

Custom Hooks

typescript

```tsx
// Extract logic into reusable hooks
function useFetch<T>(url: string): { data: T | null; loading: boolean; error: E
    const [data, setData] = useState<T | null>(null);
    const [loading, setLoading] = useState(true);
    const [error, setError] = useState<Error | null>(null);

    useEffect(() => {
        const fetchData = async () => {
            try {
                const response = await fetch(url);
                const json = await response.json();
                setData(json);
            } catch (err) {
                setError(err as Error);
            } finally {
                setLoading(false);
            }
        };

        fetchData();
    }, [url]);

    return { data, loading, error };
}

// Usage
function UserProfile() {
    const { data: user, loading, error } = useFetch<User>('/api/user');

    if (loading) return <div>Loading...</div>;
    if (error) return <div>Error: {error.message}</div>;
    return <div>{user?.name}</div>;
}
```

Follow-up Questions:

1. "What's the difference between `useCallback` and `useMemo`?"
2. "How do you prevent infinite loops in useEffect?"
3. "What's a closure in useEffect?"

Q8: Explain React Context API and State Management

Question: "Explain React Context API. When would you use it vs Redux? Show an example."

Model Answer:

Context API provides a way to pass data through component tree without prop drilling.

1. Basic Context Example

```typescript
```

```tsx
// Create context
interface ThemeContextType {
    theme: "light" | "dark";
    toggleTheme: () => void;
}

const ThemeContext = createContext<ThemeContextType | undefined>(undefined);

// Provider component
export const ThemeProvider: React.FC<{ children: React.ReactNode }> = ({ childr
    const [theme, setTheme] = useState<"light" | "dark">("light");

    const toggleTheme = () => {
        setTheme(prev => prev === "light" ? "dark" : "light");
    };

    return (
        <ThemeContext.Provider value={{ theme, toggleTheme }}>
            {children}
        </ThemeContext.Provider>
    );
};

// Custom hook to use context
export const useTheme = () => {
    const context = useContext(ThemeContext);
    if (!context) throw new Error("useTheme must be used within ThemeProvider")
    return context;
};

// Usage
function App() {
    return (
        <ThemeProvider>
            <Header />
            <MainContent />
        </ThemeProvider>
    );
}

function Header() {
    const { theme, toggleTheme } = useTheme();
    return (
```

```
        <header style={{ background: theme === "light" ? "#fff" : "#222" }}>
            <button onClick={toggleTheme}>Toggle Theme</button>
        </header>
    );
}
```

2. User Authentication Context

typescript

```tsx
interface User {
    id: string;
    name: string;
    email: string;
}

interface AuthContextType {
    user: User | null;
    isAuthenticated: boolean;
    login: (email: string, password: string) => Promise<void>;
    logout: () => void;
    isLoading: boolean;
}

const AuthContext = createContext<AuthContextType | undefined>(undefined);

export const AuthProvider: React.FC<{ children: React.ReactNode }> = ({ childre
    const [user, setUser] = useState<User | null>(null);
    const [isLoading, setIsLoading] = useState(false);

    const login = async (email: string, password: string) => {
        setIsLoading(true);
        try {
            const response = await fetch('/api/login', {
                method: 'POST',
                body: JSON.stringify({ email, password })
            });
            const userData = await response.json();
            setUser(userData);
        } finally {
            setIsLoading(false);
        }
    };

    const logout = () => {
        setUser(null);
    };

    return (
        <AuthContext.Provider value={{ user, isAuthenticated: !!user, login, lo
            {children}
        </AuthContext.Provider>
    );
```

```
};

export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) throw new Error("useAuth must be used within AuthProvider");
    return context;
};

// Usage in components
function Dashboard() {
    const { user, logout } = useAuth();

    if (!user) return <div>Not logged in</div>;

    return (
        <div>
            <h1>Welcome, {user.name}</h1>
            <button onClick={logout}>Logout</button>
        </div>
    );
}
```

Context vs Redux:

Feature	Context API	Redux
Setup	Simple	Complex (boilerplate)
Performance	Re-renders all consumers	Optimized (selectors)
Scalability	Small to medium apps	Large apps
Dev Tools	None	Redux DevTools
Learning Curve	Easy	Steep
Async Logic	useEffect + Context	Redux-Thunk/Saga

typescript

```
// ✅ Use Context for
- Theme (dark/light mode)
- Language preferences
- User authentication
- Small to medium state

// ✅ Use Redux for
- Complex application state
- Frequent state updates
- Time-travel debugging needed
- Large teams
```

3. Combining Multiple Contexts

typescript

```typescript
function AppProvider({ children }: { children: React.ReactNode }) {
    return (
        <AuthProvider>
            <ThemeProvider>
                <NotificationProvider>
                    {children}
                </NotificationProvider>
            </ThemeProvider>
        </AuthProvider>
    );
}

// In main.tsx
ReactDOM.render(
    <AppProvider>
        <App />
    </AppProvider>,
    document.getElementById('root')
);
```

Follow-up Questions:

1. "How do you optimize Context performance to prevent unnecessary re-renders?"
2. "What's the difference between Provider and Consumer?"
3. "How would you combine Context with useReducer?"

Q9: Explain Virtual DOM and React Reconciliation Algorithm

Question: "Explain how React's Virtual DOM works. What's the reconciliation algorithm and why does React use keys in lists?"

Model Answer:

Virtual DOM is React's in-memory representation of the DOM.

1. How Virtual DOM Works

typescript

```
// React creates virtual DOM from JSX
const virtualDOM = (
    <div>
        <h1>Hello</h1>
        <p>World</p>
    </div>
);

// Gets compiled to:
React.createElement("div", null,
    React.createElement("h1", null, "Hello"),
    React.createElement("p", null, "World")
);

// Creates virtual DOM tree:
{
    type: "div",
    props: { children: [...] },
    children: [
        { type: "h1", props: { children: "Hello" } },
        { type: "p", props: { children: "World" } }
    ]
}
```

2. Reconciliation (Diffing Algorithm)

React compares old and new virtual DOM trees:

typescript

```
// Initial render
const [count, setCount] = useState(0);

return (
    <div>
        <h1>Count: {count}</h1>
        <p>Click to increment</p>
        <button onClick={() => setCount(c => c + 1)}>Increment</button>
    </div>
);

// When count changes to 1
// React creates new virtual DOM
// Compares with previous virtual DOM
// Only updates the <h1> text content
// Does NOT re-create button or p tag
```

3. Why Keys Matter in Lists

typescript

```
// ❌ Without keys (problematic)
function UserList({ users }) {
    return (
        <ul>
            {users.map((user, index) => (
                <li key={index}>{user.name}</li> // BAD: using index as key
            ))}
        </ul>
    );
}

// Problem: If list is reordered, React gets confused
// Keys help React identify which items have changed

// ✅ With proper keys
function UserList({ users }) {
    return (
        <ul>
            {users.map(user => (
                <li key={user.id}>{user.name}</li> // GOOD: using stable ID
            ))}
        </ul>
    );
}

// Example: Reordering with keys
// Initial: [{ id: 1, name: "Alice" }, { id: 2, name: "Bob" }]
// After: [{ id: 2, name: "Bob" }, { id: 1, name: "Alice" }]

// With correct keys:
// React knows item with id=2 moved to top
// Reuses DOM node, only changes order

// Without keys (or with index):
// React thinks first item changed to "Bob"
// Re-renders unnecessarily
// Can cause issues with form state, animations
```

4. Reconciliation Rules

typescript

```
// Rule 1: Different types create different trees
<div /> -> <span /> // Completely new tree, unmounts div

// Rule 2: Same type, different props/children
<MyComponent foo="1" /> -> <MyComponent foo="2" /> // Updates props

// Rule 3: Key prop helps identify items
// With key: React matches items, preserves component state
// Without key: Items are matched by position

// Example: Input state with keys
function UserForm() {
    const [users, setUsers] = useState([
        { id: 1, name: "Alice" },
        { id: 2, name: "Bob" }
    ]);

    return (
        <div>
            {users.map(user => (
                <div key={user.id}> {/* Key preserves input state */}
                    <input defaultValue={user.name} />
                </div>
            ))}
        </div>
    );
}

// If you reverse users array:
// ✅ With key: Input values stay with correct user
// ❌ Without key: Input values get swapped
```

5. Preventing Unnecessary Re-renders

typescript

```typescript
// React.memo: Prevent re-render if props unchanged
const UserCard = React.memo(({ user }: { user: User }) => {
    console.log(`Rendering ${user.name}`);
    return <div>{user.name}</div>;
});

// useMemo: Memoize expensive computations
const expensiveValue = useMemo(() => {
    return complexCalculation(data);
}, [data]); // Only recalculate when data changes

// useCallback: Memoize callback functions
const handleClick = useCallback(() => {
    console.log('Clicked');
}, []); // Function identity stays same

function Parent() {
    const [count, setCount] = useState(0);

    // Without useCallback, Child re-renders on every Parent render
    const handleClick = useCallback(() => {
        setCount(c => c + 1);
    }, []);

    return <Child onClick={handleClick} />;
}
```

Follow-up Questions:

1. "What's the difference between controlled and uncontrolled components?"
2. "How does React handle event delegation?"
3. "What happens during the mount and unmount lifecycle?"

Q10: How Would You Build a Form with Validation in React?

Question: "Show me how you'd build a form with validation, error messages, and submission handling in React with TypeScript."

Model Answer:

typescript

```
// Form state and validation
interface FormData {
    email: string;
    password: string;
    confirmPassword: string;
    rememberMe: boolean;
}

interface FormErrors {
    email?: string;
    password?: string;
    confirmPassword?: string;
}

// Validation rules
const validateForm = (data: FormData): FormErrors => {
    const errors: FormErrors = {};

    // Email validation
    const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
    if (!data.email) {
        errors.email = "Email is required";
    } else if (!emailRegex.test(data.email)) {
        errors.email = "Invalid email format";
    }

    // Password validation
    if (!data.password) {
        errors.password = "Password is required";
    } else if (data.password.length < 8) {
        errors.password = "Password must be at least 8 characters";
    }

    // Confirm password
    if (data.password !== data.confirmPassword) {
        errors.confirmPassword = "Passwords do not match";
    }

    return errors;
};

// React form component
function LoginForm() {
```

```
    const [formData, setFormData] = useState<FormData>({
        email: "",
        password: "",
        confirmPassword: "",
        rememberMe: false
    });

    const [errors, setErrors] = useState<FormErrors>({});
    const [isSubmitting, setIsSubmitting] = useState(false);
    const [submitError, setSubmitError] = useState("");
    const [submitSuccess, setSubmitSuccess] = useState(false);

    // Real-time validation
    const handleInputChange = (e: React.ChangeEvent<HTMLInputElement>) => {
        const { name, value, type, checked } = e.target;

        setFormData(prev => ({
            ...prev,
            [name]: type === 'checkbox' ? checked : value
        }));

        // Clear error for this field when user starts typing
        if (errors[name as keyof FormErrors]) {
            setErrors(prev => ({
                ...prev,
                [name]: undefined
            }));
        }
    };

    const handleSubmit = async (e: React.FormEvent) => {
        e.preventDefault();
        setSubmitError("");
        setSubmitSuccess(false);

        // Validate
        const validationErrors = validateForm(formData);
        if (Object.keys(validationErrors).length > 0) {
            setErrors(validationErrors);
            return;
        }

        // Submit
        setIsSubmitting(true);
```

```
    try {
        const response = await fetch('/api/login', {
            method: 'POST',
            headers: { 'Content-Type': 'application/json' },
            body: JSON.stringify({
                email: formData.email,
                password: formData.password,
                rememberMe: formData.rememberMe
            })
        });

        if (!response.ok) {
            throw new Error('Login failed');
        }

        setSubmitSuccess(true);
        setFormData({ email: "", password: "", confirmPassword: "", remembe

    } catch (error) {
        setSubmitError(error instanceof Error ? error.message : 'Unknown er
    } finally {
        setIsSubmitting(false);
    }
};

return (
    <form onSubmit={handleSubmit} noValidate>
        {submitSuccess && (
            <div style={{ color: 'green', marginBottom: '1rem' }}>
                Login successful!
            </div>
        )}

        {submitError && (
            <div style={{ color: 'red', marginBottom: '1rem' }}>
                {submitError}
            </div>
        )}

        <div style={{ marginBottom: '1rem' }}>
            <label htmlFor="email">Email:</label>
            <input
                id="email"
                name="email"
```

```
                type="email"
                value={formData.email}
                onChange={handleInputChange}
                style={{ borderColor: errors.email ? 'red' : 'gray' }}
            />
            {errors.email && <span style={{ color: 'red' }}>{errors.email}<
        </div>

        <div style={{ marginBottom: '1rem' }}>
            <label htmlFor="password">Password:</label>
            <input
                id="password"
                name="password"
                type="password"
                value={formData.password}
                onChange={handleInputChange}
                style={{ borderColor: errors.password ? 'red' : 'gray' }}
            />
            {errors.password && <span style={{ color: 'red' }}>{errors.pass
        </div>

        <div style={{ marginBottom: '1rem' }}>
            <label htmlFor="confirmPassword">Confirm Password:</label>
            <input
                id="confirmPassword"
                name="confirmPassword"
                type="password"
                value={formData.confirmPassword}
                onChange={handleInputChange}
                style={{ borderColor: errors.confirmPassword ? 'red' : 'gra
            />
            {errors.confirmPassword && (
                <span style={{ color: 'red' }}>{errors.confirmPassword}</sp
            )}
        </div>

        <div style={{ marginBottom: '1rem' }}>
            <input
                id="rememberMe"
                name="rememberMe"
                type="checkbox"
                checked={formData.rememberMe}
                onChange={handleInputChange}
            />
```

```
                <label htmlFor="rememberMe">Remember me</label>
            </div>

            <button type="submit" disabled={isSubmitting}>
                {isSubmitting ? 'Logging in...' : 'Login'}
            </button>
        </form>
    );
}

export default LoginForm;
```

Using a Form Library (Recommended for Production)

typescript

```tsx
// Using react-hook-form (popular, performant)
import { useForm } from 'react-hook-form';
import * as yup from 'yup';
import { yupResolver } from '@hookform/resolvers/yup';

const schema = yup.object().shape({
    email: yup.string().email('Invalid email').required('Email required'),
    password: yup.string().min(8, 'Min 8 chars').required('Password required'),
    confirmPassword: yup.string()
        .oneOf([yup.ref('password')], 'Passwords must match')
        .required('Confirm password required')
});

function LoginForm() {
    const { register, handleSubmit, formState: { errors }, control } =
        useForm<FormData>({
            resolver: yupResolver(schema)
        });

    const onSubmit = async (data: FormData) => {
        const response = await fetch('/api/login', {
            method: 'POST',
            body: JSON.stringify(data)
        });
        // Handle response
    };

    return (
        <form onSubmit={handleSubmit(onSubmit)}>
            <input {...register('email')} />
            {errors.email && <span>{errors.email.message}</span>}

            <input {...register('password')} type="password" />
            {errors.password && <span>{errors.password.message}</span>}

            <button type="submit">Submit</button>
        </form>
    );
}
```

Follow-up Questions:

1. "How would you handle async validation (checking if email exists)?"

2. "What libraries would you use for form management?"
3. "How would you handle multi-step forms?"

Node.js Backend Development

Q11: Explain Middleware in Express and Create Custom Middleware

Question: "What are middleware in Express? Show me how you'd create custom middleware for authentication, logging, and error handling."

Model Answer:

Middleware are functions that process requests before they reach route handlers.

1. Basic Middleware Concept

typescript

```typescript
import express, { Request, Response, NextFunction } from 'express';

const app = express();

// Simple middleware
app.use((req: Request, res: Response, next: NextFunction) => {
    console.log(`${req.method} ${req.path}`);
    next(); // Pass control to next middleware
});

// Middleware with parameters
app.get('/users/:id', (req: Request, res: Response) => {
    res.json({ id: req.params.id });
});
```

2. Logging Middleware

typescript

```typescript
// Custom logging middleware
const loggingMiddleware = (req: Request, res: Response, next: NextFunction) =>
    const start = Date.now();

    // Capture original send function
    const originalSend = res.send;

    res.send = function(data) {
        const duration = Date.now() - start;
        console.log(`${req.method} ${req.path} - ${res.statusCode} - ${duration

        return originalSend.call(this, data);
    };

    next();
};

app.use(loggingMiddleware);
```

3. Authentication Middleware

typescript

```typescript
// Authentication middleware
interface AuthenticatedRequest extends Request {
    user?: { id: string; email: string };
}

const authMiddleware = (req: AuthenticatedRequest, res: Response, next: NextFun
    const token = req.headers.authorization?.split(' ')[1]; // Bearer <token>

    if (!token) {
        return res.status(401).json({ error: 'Missing token' });
    }

    try {
        // Verify JWT (pseudo code)
        const decoded = verifyJWT(token);
        req.user = decoded;
        next();
    } catch (error) {
        return res.status(401).json({ error: 'Invalid token' });
    }
};

// Protected route
app.get('/me', authMiddleware, (req: AuthenticatedRequest, res: Response) => {
    res.json(req.user);
});
```

4. Authorization Middleware

typescript

```typescript
// Check user roles/permissions
const authorizeRole = (allowedRoles: string[]) => {
    return (req: AuthenticatedRequest, res: Response, next: NextFunction) => {
        if (!req.user) {
            return res.status(401).json({ error: 'Not authenticated' });
        }

        // Fetch user roles from database
        const userRoles = ['user']; // Would fetch from DB

        const hasAccess = allowedRoles.some(role => userRoles.includes(role));

        if (!hasAccess) {
            return res.status(403).json({ error: 'Forbidden' });
        }

        next();
    };
};

// Admin-only route
app.delete('/admin/users/:id',
    authMiddleware,
    authorizeRole(['admin']),
    (req: AuthenticatedRequest, res: Response) => {
        res.json({ message: 'User deleted' });
    }
);
```

5. Error Handling Middleware

typescript

```typescript
// Must be last middleware (4 parameters)
app.use((error: Error, req: Request, res: Response, next: NextFunction) => {
    console.error('Error:', error);

    res.status(500).json({
        error: 'Internal server error',
        message: process.env.NODE_ENV === 'development' ? error.message : undef
    });
});

// Throwing errors in routes
app.get('/users/:id', (req: Request, res: Response, next: NextFunction) => {
    try {
        const user = getUser(req.params.id);
        if (!user) {
            throw new Error('User not found'); // Caught by error middleware
        }
        res.json(user);
    } catch (error) {
        next(error); // Pass to error middleware
    }
});
```

6. Complete Middleware Chain

typescript

```typescript
// Order matters! Middleware execute in order
app.use(express.json()); // Parse JSON body
app.use(loggingMiddleware); // Log requests
app.use(authMiddleware); // Authenticate
app.use(authorizeRole(['user', 'admin'])); // Authorize

// Routes
app.get('/users', (req: Request, res: Response) => {
    res.json({ users: [] });
});

// Error handling (must be last)
app.use((error: Error, req: Request, res: Response, next: NextFunction) => {
    res.status(500).json({ error: error.message });
});
```

7. Rate Limiting Middleware

typescript

```typescript
interface RateLimitStore {
    [ip: string]: { count: number; resetTime: number };
}

const rateLimitMiddleware = (maxRequests: number = 100, windowMs: number = 6000
    const store: RateLimitStore = {};

    return (req: Request, res: Response, next: NextFunction) => {
        const ip = req.ip || 'unknown';
        const now = Date.now();

        if (!store[ip]) {
            store[ip] = { count: 0, resetTime: now + windowMs };
        }

        if (now > store[ip].resetTime) {
            store[ip] = { count: 0, resetTime: now + windowMs };
        }

        store[ip].count++;

        if (store[ip].count > maxRequests) {
            return res.status(429).json({ error: 'Too many requests' });
        }

        next();
    };
};

app.use(rateLimitMiddleware(100, 60000)); // 100 requests per minute
```

Follow-up Questions:

1. "What's the difference between app.use() and app.get(middleware)?"
2. "How do you handle async middleware?"
3. "How do you compose multiple middleware together?"

Q12: Explain Async/Await Error Handling in Node.js

Question: "Show me how you'd properly handle errors with async/await in Node.js. What's the best pattern?"

Model Answer:

typescript

```
// Express async route handler wrapper
const asyncHandler = (fn: Function) => {
    return (req: Request, res: Response, next: NextFunction) => {
        Promise.resolve(fn(req, res, next)).catch(next);
    };
};

// Usage
app.get('/users/:id', asyncHandler(async (req: Request, res: Response) => {
    const user = await db.getUser(req.params.id); // If this throws, caught aut
    if (!user) throw new Error('User not found');
    res.json(user);
}));

// Without wrapper (error prone)
app.get('/users/:id', async (req: Request, res: Response) => {
    try {
        const user = await db.getUser(req.params.id);
        res.json(user);
    } catch (error) {
        next(error); // Must manually pass to error handler
    }
});

// Better: Try-catch pattern
async function getUser(userId: string): Promise<User> {
    try {
        const response = await fetch(`/api/users/${userId}`);

        if (!response.ok) {
            throw new Error(`HTTP error! status: ${response.status}`);
        }

        return await response.json();
    } catch (error) {
        if (error instanceof TypeError) {
            throw new Error('Network error');
        }
        throw error;
    }
}

// Real-world example
```

```
app.post('/users', asyncHandler(async (req: Request, res: Response) => {
    const { email, password } = req.body;

    // Validation
    if (!email || !password) {
        throw new Error('Email and password required');
    }

    // Check if user exists
    const existingUser = await User.findOne({ email });
    if (existingUser) {
        throw new Error('User already exists');
    }

    // Create user
    const hashedPassword = await bcrypt.hash(password, 10);
    const user = await User.create({
        email,
        password: hashedPassword
    });

    res.status(201).json(user);
}));

// Error middleware catches all errors
app.use((error: any, req: Request, res: Response, next: NextFunction) => {
    const statusCode = error.statusCode || 500;
    const message = error.message || 'Internal server error';

    console.error('Error:', message);

    res.status(statusCode).json({ error: message });
});
```

Q13: Design a RESTful API with Proper Status Codes and Error Handling

Question: "Design a RESTful API for a blog. Include proper HTTP status codes, error responses, and pagination."

Model Answer:

typescript

```typescript
import express, { Request, Response, NextFunction } from 'express';

// Types
interface Post {
    id: string;
    title: string;
    content: string;
    authorId: string;
    createdAt: Date;
}

interface ApiResponse<T> {
    data?: T;
    error?: string;
    pagination?: { page: number; limit: number; total: number };
}

const app = express();
app.use(express.json());

// POST: Create a new post
// 201: Created
// 400: Bad Request
// 401: Unauthorized
app.post('/posts', async (req: Request, res: Response<ApiResponse<Post>>) => {
    const { title, content } = req.body;

    // Validation
    if (!title || !content) {
        return res.status(400).json({
            error: 'Title and content are required'
        });
    }

    try {
        const post: Post = {
            id: `post-${Date.now()}`,
            title,
            content,
            authorId: req.user?.id || '',
            createdAt: new Date()
        };
```

```
        // Save to database
        await db.posts.insert(post);

        res.status(201).json({ data: post });
    } catch (error) {
        res.status(500).json({ error: 'Failed to create post' });
    }
});

// GET: List posts with pagination
// 200: OK
// 400: Bad Request (invalid pagination params)
app.get('/posts', async (req: Request, res: Response<ApiResponse<Post[]>>) => {
    const page = parseInt(req.query.page as string) || 1;
    const limit = parseInt(req.query.limit as string) || 10;

    // Validation
    if (page < 1 || limit < 1 || limit > 100) {
        return res.status(400).json({
            error: 'Invalid pagination parameters'
        });
    }

    try {
        const skip = (page - 1) * limit;
        const posts = await db.posts.find({}, { skip, limit });
        const total = await db.posts.count();

        res.status(200).json({
            data: posts,
            pagination: { page, limit, total }
        });
    } catch (error) {
        res.status(500).json({ error: 'Failed to fetch posts' });
    }
});

// GET: Get single post
// 200: OK
// 404: Not Found
app.get('/posts/:id', async (req: Request, res: Response<ApiResponse<Post>>) =>
    try {
        const post = await db.posts.findOne({ id: req.params.id });
```

```
        if (!post) {
            return res.status(404).json({ error: 'Post not found' });
        }

        res.status(200).json({ data: post });
    } catch (error) {
        res.status(500).json({ error: 'Failed to fetch post' });
    }
});

// PATCH: Update post
// 200: OK
// 404: Not Found
// 400: Bad Request
// 403: Forbidden (not author)
app.patch('/posts/:id', async (req: Request, res: Response<ApiResponse<Post>>)
    const { title, content } = req.body;

    if (!title && !content) {
        return res.status(400).json({
            error: 'At least one field (title or content) is required'
        });
    }

    try {
        const post = await db.posts.findOne({ id: req.params.id });

        if (!post) {
            return res.status(404).json({ error: 'Post not found' });
        }

        // Authorization check
        if (post.authorId !== req.user?.id) {
            return res.status(403).json({ error: 'Not authorized to update this
        }

        // Update
        const updatedPost = {
            ...post,
            ...(title && { title }),
            ...(content && { content })
        };

        await db.posts.updateOne({ id: req.params.id }, updatedPost);
```

```
        res.status(200).json({ data: updatedPost });
    } catch (error) {
        res.status(500).json({ error: 'Failed to update post' });
    }
});

// DELETE: Delete post
// 204: No Content
// 404: Not Found
// 403: Forbidden
app.delete('/posts/:id', async (req: Request, res: Response) => {
    try {
        const post = await db.posts.findOne({ id: req.params.id });

        if (!post) {
            return res.status(404).json({ error: 'Post not found' });
        }

        if (post.authorId !== req.user?.id) {
            return res.status(403).json({ error: 'Not authorized' });
        }

        await db.posts.deleteOne({ id: req.params.id });

        res.status(204).send(); // No content
    } catch (error) {
        res.status(500).json({ error: 'Failed to delete post' });
    }
});

// HTTP Status Code Summary
const STATUS_CODES = {
    // 2xx - Success
    200: 'OK - Request succeeded',
    201: 'Created - Resource created',
    204: 'No Content - Success, no response body',

    // 4xx - Client Error
    400: 'Bad Request - Invalid input',
    401: 'Unauthorized - Authentication required',
    403: 'Forbidden - Authorized but denied',
    404: 'Not Found - Resource doesn\'t exist',
    409: 'Conflict - Resource conflict (e.g., duplicate)',
```

```
    422: 'Unprocessable Entity - Validation failed',
    429: 'Too Many Requests - Rate limited',

    // 5xx - Server Error
    500: 'Internal Server Error',
    503: 'Service Unavailable'
};

// Error handling middleware
app.use((error: Error, req: Request, res: Response, next: NextFunction) => {
    console.error(error);
    res.status(500).json({ error: 'Internal server error' });
});
```

Follow-up Questions:

1. "How would you handle versioning in APIs (v1, v2)?"
2. "How would you document an API (Swagger/OpenAPI)?"
3. "What's idempotency and why does it matter?"

Serverless Architecture & AWS

Q14: Explain Serverless Architecture and AWS Lambda

Question: "Explain serverless architecture. What are the benefits and drawbacks? How have you used AWS Lambda?"

Model Answer:

Serverless Architecture means deploying code without managing servers. AWS Lambda is a compute service that runs code in response to events.

1. How AWS Lambda Works

typescript

```typescript
// Lambda handler function
export const handler = async (event: any, context: any) => {
    try {
        console.log('Event:', event);
        console.log('Context:', context);

        // Your business logic
        const result = {
            statusCode: 200,
            body: JSON.stringify({ message: 'Hello from Lambda!' })
        };

        return result;
    } catch (error) {
        return {
            statusCode: 500,
            body: JSON.stringify({ error: error.message })
        };
    }
};
```

2. Lambda with API Gateway

typescript

```
// HTTP request from API Gateway
export const handleHttpRequest = async (event: any) => {
    const { body, pathParameters, queryStringParameters } = event;

    try {
        if (event.httpMethod === 'POST') {
            const data = JSON.parse(body);
            const result = await createUser(data);

            return {
                statusCode: 201,
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(result)
            };
        }

        if (event.httpMethod === 'GET') {
            const userId = pathParameters?.id;
            const user = await getUser(userId);

            return {
                statusCode: 200,
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(user)
            };
        }
    } catch (error) {
        return {
            statusCode: 500,
            body: JSON.stringify({ error: error.message })
        };
    }
};
```

3. Lambda with DynamoDB

typescript

```typescript
import AWS from 'aws-sdk';

const dynamodb = new AWS.DynamoDB.DocumentClient();

export const saveUserData = async (event: any) => {
    const { userId, userData } = event;

    try {
        // Write to DynamoDB
        await dynamodb.put({
            TableName: 'Users',
            Item: {
                userId,
                ...userData,
                createdAt: new Date().toISOString()
            }
        }).promise();

        return { statusCode: 200, message: 'User saved' };
    } catch (error) {
        console.error('Error:', error);
        return { statusCode: 500, error: error.message };
    }
};

export const getUserData = async (userId: string) => {
    try {
        const result = await dynamodb.get({
            TableName: 'Users',
            Key: { userId }
        }).promise();

        return result.Item;
    } catch (error) {
        throw new Error(`Failed to get user: ${error}`);
    }
};
```

4. Lambda with S3

typescript

```typescript
import AWS from 'aws-sdk';

const s3 = new AWS.S3();

export const uploadFileToS3 = async (event: any) => {
    const { fileName, fileContent } = event;

    try {
        const params = {
            Bucket: 'my-bucket',
            Key: `uploads/${fileName}`,
            Body: fileContent,
            ContentType: 'text/plain'
        };

        const result = await s3.upload(params).promise();

        return {
            statusCode: 200,
            location: result.Location
        };
    } catch (error) {
        return { statusCode: 500, error: error.message };
    }
};

export const processS3Event = async (event: any) => {
    // Triggered when file uploaded to S3
    const bucket = event.Records[0].s3.bucket.name;
    const key = event.Records[0].s3.object.key;

    console.log(`Processing file: s3://${bucket}/${key}`);

    try {
        const file = await s3.getObject({ Bucket: bucket, Key: key }).promise()
        // Process file
        return { statusCode: 200, message: 'File processed' };
    } catch (error) {
        return { statusCode: 500, error: error.message };
    }
};
```

5. Lambda with SQS (Async Processing)

typescript

```typescript
import AWS from 'aws-sdk';

const sqs = new AWS.SQS();
const lambda = new AWS.Lambda();

// Send job to queue
export const sendJobToQueue = async (jobData: any) => {
    try {
        await sqs.sendMessage({
            QueueUrl: process.env.QUEUE_URL!,
            MessageBody: JSON.stringify(jobData)
        }).promise();

        return { statusCode: 200, message: 'Job queued' };
    } catch (error) {
        return { statusCode: 500, error: error.message };
    }
};

// Process messages from queue
export const processQueueMessages = async (event: any) => {
    const records = event.Records;

    for (const record of records) {
        try {
            const message = JSON.parse(record.body);
            console.log('Processing message:', message);

            // Do work
            await heavyComputation(message);

            // Delete from queue
            await sqs.deleteMessage({
                QueueUrl: process.env.QUEUE_URL!,
                ReceiptHandle: record.receiptHandle
            }).promise();
        } catch (error) {
            console.error('Error processing message:', error);
            // Message will be retried
        }
    }
};
```

6. Serverless Framework Example

yaml

```yaml
# serverless.yml
service: my-api

provider:
  name: aws
  runtime: nodejs18.x
  region: us-east-1
  environment:
    DYNAMODB_TABLE: Users
    SQS_QUEUE_URL: ${self:custom.sqsQueueUrl}

functions:
  createUser:
    handler: src/handlers/users.create
    events:
      - http:
          path: users
          method: post
          cors: true
    environment:
      DATABASE_URL: ${ssm:/database/url}

  getUser:
    handler: src/handlers/users.get
    events:
      - http:
          path: users/{id}
          method: get

  processFile:
    handler: src/handlers/s3.process
    events:
      - s3:
          bucket: my-bucket
          event: s3:ObjectCreated:*
          rules:
            - prefix: uploads/

  processQueue:
    handler: src/handlers/queue.process
    events:
      - sqs:
          arn:
```

```
            Fn::GetAtt: [MyQueue, Arn]
          batchSize: 10

resources:
  Resources:
    MyQueue:
      Type: AWS::SQS::Queue
      Properties:
        QueueName: my-queue
        VisibilityTimeout: 300
```

Serverless Benefits vs Drawbacks:

Benefit	Drawback
✅ No server management	❌ Cold starts (~1-3 seconds)
✅ Auto-scaling	❌ Limited execution time (15 min)
✅ Pay-per-use	❌ Vendor lock-in
✅ High availability	❌ Debugging harder
✅ Rapid deployment	❌ Stateless (can't keep connections)

Follow-up Questions:

1. "How do you handle cold starts in Lambda?"
2. "What are Lambda layers and when would you use them?"
3. "How would you structure a serverless monolith vs microservices?"

Q15: Explain AWS API Gateway, SQS, and SNS

Question: "Explain AWS API Gateway, SQS, and SNS. When would you use each? Show integration examples."

Model Answer:

AWS API Gateway - Frontend for APIs
AWS SQS - Message queue for asynchronous processing
AWS SNS - Pub/Sub notification service

1. API Gateway

typescript

```typescript
// API Gateway → Lambda
export const handleHttpRequest = async (event: any) => {
    const { httpMethod, path, body, queryStringParameters } = event;

    console.log(`${httpMethod} ${path}`);

    try {
        if (path === '/users' && httpMethod === 'POST') {
            const user = JSON.parse(body);
            const createdUser = await createUser(user);

            return {
                statusCode: 201,
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(createdUser)
            };
        }

        if (path.startsWith('/users/') && httpMethod === 'GET') {
            const id = path.split('/')[2];
            const user = await getUser(id);

            return {
                statusCode: 200,
                headers: { 'Content-Type': 'application/json' },
                body: JSON.stringify(user)
            };
        }

        return { statusCode: 404, body: JSON.stringify({ error: 'Not found' }) }
    } catch (error) {
        return { statusCode: 500, body: JSON.stringify({ error: error.message }) }
    }
};
```

2. SQS - Asynchronous Processing

typescript

```
import AWS from 'aws-sdk';

const sqs = new AWS.SQS();

// Producer: Send email job to queue
export const sendEmailAsync = async (email: string, subject: string) => {
    try {
        await sqs.sendMessage({
            QueueUrl: process.env.EMAIL_QUEUE_URL!,
            MessageBody: JSON.stringify({ email, subject }),
            MessageGroupId: email // For FIFO queue (ordered)
        }).promise();

        return { statusCode: 200, message: 'Email queued' };
    } catch (error) {
        throw new Error(`Failed to queue email: ${error}`);
    }
};

// Consumer: Process emails from queue
export const processEmailQueue = async (event: any) => {
    for (const record of event.Records) {
        try {
            const { email, subject } = JSON.parse(record.body);

            console.log(`Sending email to ${email}`);
            await sendEmailViaSES(email, subject);

            // Delete from queue on success
            await sqs.deleteMessage({
                QueueUrl: process.env.EMAIL_QUEUE_URL!,
                ReceiptHandle: record.receiptHandle
            }).promise();
        } catch (error) {
            console.error('Error processing email:', error);
            // Will be retried based on visibility timeout
        }
    }
};

// Batch sending with SQS
export const batchSendMessages = async (messages: any[]) => {
    const entries = messages.map((msg, i) => ({
```

```
        Id: String(i),
        MessageBody: JSON.stringify(msg),
        MessageGroupId: 'group1'
    }));

    await sqs.sendMessageBatch({
        QueueUrl: process.env.QUEUE_URL!,
        Entries: entries
    }).promise();
};
```

3. SNS - Pub/Sub Notifications

typescript

```
import AWS from 'aws-sdk';

const sns = new AWS.SNS();

// Publisher: Publish order events
export const publishOrderEvent = async (order: any) => {
    try {
        await sns.publish({
            TopicArn: process.env.ORDER_TOPIC_ARN!,
            Subject: 'New Order Created',
            Message: JSON.stringify({
                orderId: order.id,
                userId: order.userId,
                amount: order.amount,
                timestamp: new Date().toISOString()
            }),
            MessageAttributes: {
                eventType: { DataType: 'String', StringValue: 'order.created' }
            }
        }).promise();

        return { statusCode: 200, message: 'Event published' };
    } catch (error) {
        throw new Error(`Failed to publish event: ${error}`);
    }
};

// Subscriber 1: Email service (Lambda)
export const sendOrderConfirmationEmail = async (event: any) => {
    const message = JSON.parse(event.Records[0].Sns.Message);
    console.log('Received order event:', message);

    await sendEmail({
        email: message.userId,
        subject: 'Order Confirmation',
        body: `Your order ${message.orderId} for $${message.amount} has been re
    });
};

// Subscriber 2: Analytics service (Lambda)
export const trackOrderAnalytics = async (event: any) => {
    const message = JSON.parse(event.Records[0].Sns.Message);
    console.log('Recording analytics for order:', message.orderId);
```

```
    await recordAnalytics({
        eventType: 'order_created',
        orderId: message.orderId,
        amount: message.amount
    });
};

// Subscriber 3: Inventory service (SQS)
// SNS can fan-out to SQS for decoupling
```

4. Event-Driven Architecture Example

typescript

```
// API endpoint receives request
// POST /orders
export const createOrder = async (event: any) => {
    try {
        const order = JSON.parse(event.body);

        // Save to database
        const savedOrder = await db.orders.insert(order);

        // Publish event (triggers all subscribers)
        await sns.publish({
            TopicArn: process.env.ORDER_TOPIC_ARN!,
            Message: JSON.stringify({
                action: 'ORDER_CREATED',
                orderId: savedOrder.id,
                userId: order.userId,
                amount: order.amount
            })
        }).promise();

        return {
            statusCode: 201,
            body: JSON.stringify(savedOrder)
        };
    } catch (error) {
        return { statusCode: 500, body: JSON.stringify({ error: error.message }) };
    }
};

// SNS Topic → Multiple Subscribers
// ├── Email Service (sends confirmation)
// ├── Analytics Service (records event)
// ├── Inventory Service (via SQS)
// └── Notification Service (sends SMS)
```

Comparison:

Feature	SQS	SNS
Type	Queue (1 → 1)	Pub/Sub (1 → Many)
Delivery	Pull model	Push model
Use	Decoupling, rate limiting	Fan-out, notifications
Retention	4 days - 14 days	Immediate delivery
Ordering	FIFO optional	No ordering

Follow-up Questions:

1. "What's the difference between SNS and SES?"
2. "How would you implement retry logic with SQS?"
3. "How do you handle dead-letter queues?"

API Design

Q16: Explain RESTful APIs vs GraphQL APIs

Question: "Compare RESTful and GraphQL APIs. When would you use each? Show examples of both."

Model Answer:

REST - Resource-based architecture
GraphQL - Query-based architecture

1. REST API Example

```typescript
```

```
// Multiple endpoints for different resources
GET /api/users
GET /api/users/:id
GET /api/users/:id/posts
GET /api/users/:id/posts/:postId/comments

// Over-fetching: Get more data than needed
GET /api/users/1
{
    id: 1,
    name: "John",
    email: "john@example.com",
    phone: "123-456-7890",
    address: "123 Main St",
    // ... many more fields you don't need
}

// Under-fetching: Need multiple requests
GET /api/users/1 // Get user
GET /api/users/1/posts // Get posts
GET /api/posts/1/comments // Get comments
// 3 requests!
```

2. GraphQL API Example

graphql

```
# Single endpoint
POST /graphql

# Query only what you need
query {
  user(id: 1) {
    id
    name
    email
    posts {
      id
      title
      comments {
        id
        text
      }
    }
  }
}

# Response matches exactly what you requested
{
  "data": {
    "user": {
      "id": 1,
      "name": "John",
      "email": "john@example.com",
      "posts": [
        {
          "id": 1,
          "title": "My First Post",
          "comments": [
            { "id": 1, "text": "Great post!" }
          ]
        }
      ]
    }
  }
}
```

3. GraphQL Server Implementation

typescript

```
import { ApolloServer, gql } from 'apollo-server-express';
import express from 'express';

const app = express();

// Schema definition
const typeDefs = gql`
  type User {
    id: ID!
    name: String!
    email: String!
    posts: [Post!]!
  }

  type Post {
    id: ID!
    title: String!
    content: String!
    author: User!
    comments: [Comment!]!
  }

  type Comment {
    id: ID!
    text: String!
    author: User!
  }

  type Query {
    user(id: ID!): User
    users: [User!]!
    post(id: ID!): Post
    posts: [Post!]!
  }

  type Mutation {
    createUser(name: String!, email: String!): User!
    createPost(title: String!, content: String!, authorId: ID!): Post!
    createComment(text: String!, postId: ID!, authorId: ID!): Comment!
  }
`;

// Resolvers
```

```
const resolvers = {
    Query: {
        user: async (_, { id }) => {
            return await db.users.findById(id);
        },
        users: async () => {
            return await db.users.find();
        },
        post: async (_, { id }) => {
            return await db.posts.findById(id);
        }
    },

    Mutation: {
        createUser: async (_, { name, email }) => {
            const user = { id: 'new-id', name, email };
            await db.users.insert(user);
            return user;
        },
        createPost: async (_, { title, content, authorId }) => {
            const post = {
                id: 'new-id',
                title,
                content,
                authorId,
                createdAt: new Date()
            };
            await db.posts.insert(post);
            return post;
        }
    },

    User: {
        posts: async (user) => {
            return await db.posts.find({ authorId: user.id });
        }
    },

    Post: {
        author: async (post) => {
            return await db.users.findById(post.authorId);
        },
        comments: async (post) => {
            return await db.comments.find({ postId: post.id });
```

```
        }
    },

    Comment: {
        author: async (comment) => {
            return await db.users.findById(comment.authorId);
        }
    }
};

// Create Apollo Server
const server = new ApolloServer({ typeDefs, resolvers });

await server.start();
server.applyMiddleware({ app });

app.listen(4000, () => {
    console.log('Server running at http://localhost:4000/graphql');
});
```

4. GraphQL with Mutations

typescript

```
// Update user
mutation {
  createUser(name: "Alice", email: "alice@example.com") {
    id
    name
    email
  }
}

// Create post
mutation {
  createPost(
    title: "GraphQL Basics"
    content: "GraphQL is a query language..."
    authorId: "1"
  ) {
    id
    title
    author {
      name
    }
  }
}
```

Comparison:

Feature	REST	GraphQL
Architecture	Resource-based	Query-based
Endpoints	Many	Single
Over-fetching	Yes	No
Under-fetching	Yes	No
Caching	Easy (HTTP)	Complex
Learning Curve	Easy	Steep
Real-time	Polling/WebSocket	Subscriptions
Tools	Postman	GraphQL Playground

When to Use:

```typescript
// ✅ REST for:
- Simple APIs
- File uploads
- Caching important
- Stateless operations

// ✅ GraphQL for:
- Complex data relationships
- Multiple clients (web, mobile, etc.)
- Flexible queries needed
- Real-time data subscriptions
```

Follow-up Questions:

1. "How do you handle authentication in GraphQL?"
2. "What are GraphQL subscriptions?"
3. "How do you prevent N+1 query problems in GraphQL?"

Q17: Explain OpenAPI Specification and API Documentation

Question: "What is OpenAPI specification? How would you document a REST API? Show an example."

Model Answer:

OpenAPI (formerly Swagger) is a standard for describing REST APIs.

1. OpenAPI 3.0 YAML Example

```yaml
```

```yaml
openapi: 3.0.0
info:
  title: Blog API
  description: REST API for managing blog posts
  version: 1.0.0
  contact:
    name: Support
    email: support@example.com

servers:
  - url: https://api.example.com
    description: Production
  - url: https://staging.example.com
    description: Staging

paths:
  /posts:
    get:
      summary: List all posts
      description: Retrieve a paginated list of blog posts
      operationId: listPosts
      tags:
        - Posts
      parameters:
        - name: page
          in: query
          description: Page number
          required: false
          schema:
            type: integer
            default: 1
        - name: limit
          in: query
          description: Items per page
          required: false
          schema:
            type: integer
            default: 10
            maximum: 100
      responses:
        '200':
          description: List of posts
          content:
```

```yaml
          application/json:
            schema:
              type: object
              properties:
                data:
                  type: array
                  items:
                    $ref: '#/components/schemas/Post'
                pagination:
                  $ref: '#/components/schemas/Pagination'
      '400':
        description: Invalid parameters
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Error'

  post:
    summary: Create a new post
    operationId: createPost
    tags:
      - Posts
    security:
      - bearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PostInput'
    responses:
      '201':
        description: Post created
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Post'
      '400':
        description: Invalid input
      '401':
        description: Unauthorized

/posts/{id}:
  parameters:
```

```yaml
    - name: id
      in: path
      required: true
      schema:
        type: string

  get:
    summary: Get a post by ID
    operationId: getPost
    tags:
      - Posts
    responses:
      '200':
        description: Post found
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/Post'
      '404':
        description: Post not found

  patch:
    summary: Update a post
    operationId: updatePost
    tags:
      - Posts
    security:
      - bearerAuth: []
    requestBody:
      required: true
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/PostInput'
    responses:
      '200':
        description: Post updated
      '403':
        description: Not authorized to update
      '404':
        description: Post not found

  delete:
    summary: Delete a post
```

```yaml
      operationId: deletePost
      tags:
        - Posts
      security:
        - bearerAuth: []
      responses:
        '204':
          description: Post deleted
        '403':
          description: Not authorized
        '404':
          description: Post not found

components:
  schemas:
    Post:
      type: object
      required:
        - id
        - title
        - content
        - author
        - createdAt
      properties:
        id:
          type: string
          example: "post-123"
        title:
          type: string
          example: "Getting Started with GraphQL"
        content:
          type: string
          example: "GraphQL is a query language..."
        author:
          $ref: '#/components/schemas/User'
        createdAt:
          type: string
          format: date-time

    PostInput:
      type: object
      required:
        - title
        - content
```

```yaml
    properties:
      title:
        type: string
        minLength: 5
        maxLength: 200
      content:
        type: string
        minLength: 10

  User:
    type: object
    properties:
      id:
        type: string
      name:
        type: string
      email:
        type: string
        format: email

  Pagination:
    type: object
    properties:
      page:
        type: integer
      limit:
        type: integer
      total:
        type: integer

  Error:
    type: object
    properties:
      error:
        type: string
      message:
        type: string

securitySchemes:
  bearerAuth:
    type: http
    scheme: bearer
    bearerFormat: JWT
```

typescript

```typescript
import swaggerUi from 'swagger-ui-express';
import YAML from 'yamljs';

const swaggerDocument = YAML.load('./openapi.yaml');

app.use('/api-docs', swaggerUi.serve);
app.get('/api-docs', swaggerUi.setup(swaggerDocument));

// API docs available at /api-docs
```

3. Generating OpenAPI from Code (Decorators)

typescript

```
import { Swagger } from '@nestjs/swagger';

@Swagger()
@Controller('posts')
export class PostsController {
    @Get()
    @ApiOperation({ summary: 'List all posts' })
    @ApiResponse({ status: 200, description: 'Posts found' })
    @ApiQuery({ name: 'page', required: false, type: Number })
    @ApiQuery({ name: 'limit', required: false, type: Number })
    async listPosts(@Query() query: any) {
        // Implementation
    }

    @Post()
    @ApiOperation({ summary: 'Create a post' })
    @ApiBody({ type: CreatePostDto })
    @ApiResponse({ status: 201, description: 'Post created' })
    @ApiBearerAuth()
    async createPost(@Body() dto: CreatePostDto) {
        // Implementation
    }

    @Get(':id')
    @ApiOperation({ summary: 'Get a post' })
    @ApiParam({ name: 'id', description: 'Post ID' })
    @ApiResponse({ status: 200, description: 'Post found' })
    @ApiResponse({ status: 404, description: 'Post not found' })
    async getPost(@Param('id') id: string) {
        // Implementation
    }
}
```

Benefits of OpenAPI:

- ✅ Auto-generated documentation
- ✅ Client SDK generation
- ✅ Testing tools integration
- ✅ Validation of requests/responses
- ✅ Standardized contract between backend/frontend

Follow-up Questions:

1. "How do you version APIs?"
2. "What's the difference between OpenAPI 3.0 and 3.1?"
3. "How do you handle breaking changes in APIs?"

Docker & Kubernetes

Q18: Explain Docker and Container Best Practices

Question: "Explain Docker and containers. Show a Dockerfile example for a Node.js application. What are best practices?"

Model Answer:

Docker packages your application and dependencies into a container.

1. Basic Dockerfile

```dockerfile
```

```dockerfile
# Use official Node.js image
FROM node:18-alpine

# Set working directory
WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm install --production

# Copy source code
COPY . .

# Expose port
EXPOSE 3000

# Health check
HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD node healthcheck.js

# Start application
CMD ["node", "dist/index.js"]
```

2. Multi-Stage Build (Best Practice)

dockerfile

```dockerfile
# Build stage
FROM node:18-alpine AS builder

WORKDIR /build
COPY package*.json ./
RUN npm install

COPY . .
RUN npm run build

# Production stage (smaller image)
FROM node:18-alpine

WORKDIR /app
COPY package*.json ./
RUN npm install --production

# Copy built app from builder
COPY --from=builder /build/dist ./dist

EXPOSE 3000

CMD ["node", "dist/index.js"]
```

3. .dockerignore (Similar to .gitignore)

```
node_modules
npm-debug.log
.git
.gitignore
README.md
.env
.DS_Store
dist
coverage
.vscode
```

4. Docker Best Practices

dockerfile

```
# ❌ DON'T: Run as root
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "index.js"]

# ✅ DO: Create non-root user
FROM node:18-alpine

WORKDIR /app

# Create app user
RUN addgroup -g 1001 -S nodejs
RUN adduser -S nodejs -u 1001

COPY --chown=nodejs:nodejs package*.json ./
RUN npm install --production

COPY --chown=nodejs:nodejs . .

USER nodejs

EXPOSE 3000
HEALTHCHECK --interval=30s CMD node healthcheck.js
CMD ["node", "dist/index.js"]
```

5. docker-compose for Development

yaml

```yaml
version: '3.8'

services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      NODE_ENV: development
      DATABASE_URL: postgresql://user:pass@db:5432/mydb
    volumes:
      - .:/app
      - /app/node_modules
    depends_on:
      db:
        condition: service_healthy
    networks:
      - app-network

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
      POSTGRES_DB: mydb
    ports:
      - "5432:5432"
    volumes:
      - db-data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d mydb"]
      interval: 10s
      timeout: 5s
      retries: 5
    networks:
      - app-network

volumes:
  db-data:

networks:
```

```
  app-network:
    driver: bridge
```

Follow-up Questions:

1. "What's the difference between Docker and virtual machines?"
2. "How do you optimize Docker image size?"
3. "What are Docker layers and how do caches work?"

Q19: Explain Kubernetes Basics and Deployment

Question: "Explain Kubernetes basics. How would you deploy a containerized application? Show a YAML example."

Model Answer:

Kubernetes orchestrates containerized applications at scale.

1. Kubernetes Deployment

yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: blog-api
  namespace: default
  labels:
    app: blog-api
    version: v1

spec:
  replicas: 3  # Run 3 instances
  strategy:
    type: RollingUpdate  # Gradual update
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0  # Zero downtime

  selector:
    matchLabels:
      app: blog-api

  template:
    metadata:
      labels:
        app: blog-api

    spec:
      # Security context
      securityContext:
        runAsNonRoot: true
        runAsUser: 1000

      containers:
        - name: blog-api
          image: myregistry.azurecr.io/blog-api:1.0.0
          imagePullPolicy: Always

          # Port
          ports:
            - containerPort: 3000
              name: http

          # Environment variables
```

```
        env:
          - name: NODE_ENV
            value: "production"
          - name: DATABASE_URL
            valueFrom:
              secretKeyRef:
                name: db-secret
                key: url
          - name: LOG_LEVEL
            valueFrom:
              configMapKeyRef:
                name: app-config
                key: log-level

        # Resources
        resources:
          requests:
            memory: "256Mi"
            cpu: "250m"
          limits:
            memory: "512Mi"
            cpu: "500m"

        # Health checks
        livenessProbe:
          httpGet:
            path: /health
            port: 3000
          initialDelaySeconds: 30
          periodSeconds: 10

        readinessProbe:
          httpGet:
            path: /ready
            port: 3000
          initialDelaySeconds: 5
          periodSeconds: 5

        # Logging
        volumeMounts:
          - name: logs
            mountPath: /var/logs

      volumes:
```

```yaml
        - name: logs
          emptyDir: {}

      # Pod disruption budget
      affinity:
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
            - weight: 100
              podAffinityTerm:
                labelSelector:
                  matchExpressions:
                    - key: app
                      operator: In
                      values:
                        - blog-api
                topologyKey: kubernetes.io/hostname
```

2. Service (Expose Deployment)

yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: blog-api-service
  labels:
    app: blog-api

spec:
  type: LoadBalancer  # Exposes externally
  # type: ClusterIP  # Internal only
  # type: NodePort   # Via node port

  selector:
    app: blog-api

  ports:
    - port: 80          # External port
      targetPort: 3000  # Container port
      protocol: TCP
      name: http
```

3. ConfigMap & Secret

yaml

```yaml
# ConfigMap - non-sensitive config
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config

data:
  log-level: "info"
  max-connections: "100"
  cache-ttl: "3600"

---

# Secret - sensitive data (base64 encoded)
apiVersion: v1
kind: Secret
metadata:
  name: db-secret

type: Opaque
data:
  url: cG9zdGdyZXM6Ly91c2VyOnBhc3NAZGI6NTQzMi9teWRi  # Base64 encoded
  password: bXlwYXNzd29yZA==
```

4. Ingress (API Gateway)

yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: blog-api-ingress
  annotations:
    cert-manager.io/cluster-issuer: "letsencrypt-prod"

spec:
  ingressClassName: nginx

  tls:
    - hosts:
        - api.example.com
      secretName: blog-api-tls

  rules:
    - host: api.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: blog-api-service
                port:
                  number: 80
```

5. Deploy to Kubernetes

bash

```
# Create namespace
kubectl create namespace prod

# Apply manifests
kubectl apply -f deployment.yaml -n prod
kubectl apply -f service.yaml -n prod
kubectl apply -f ingress.yaml -n prod

# Check status
kubectl get pods -n prod
kubectl get svc -n prod
kubectl logs -f deployment/blog-api -n prod

# Scaling
kubectl scale deployment blog-api --replicas=5 -n prod

# Rolling update
kubectl set image deployment/blog-api \
  blog-api=myregistry.azurecr.io/blog-api:2.0.0 -n prod

# Rollback
kubectl rollout undo deployment/blog-api -n prod
```

Follow-up Questions:

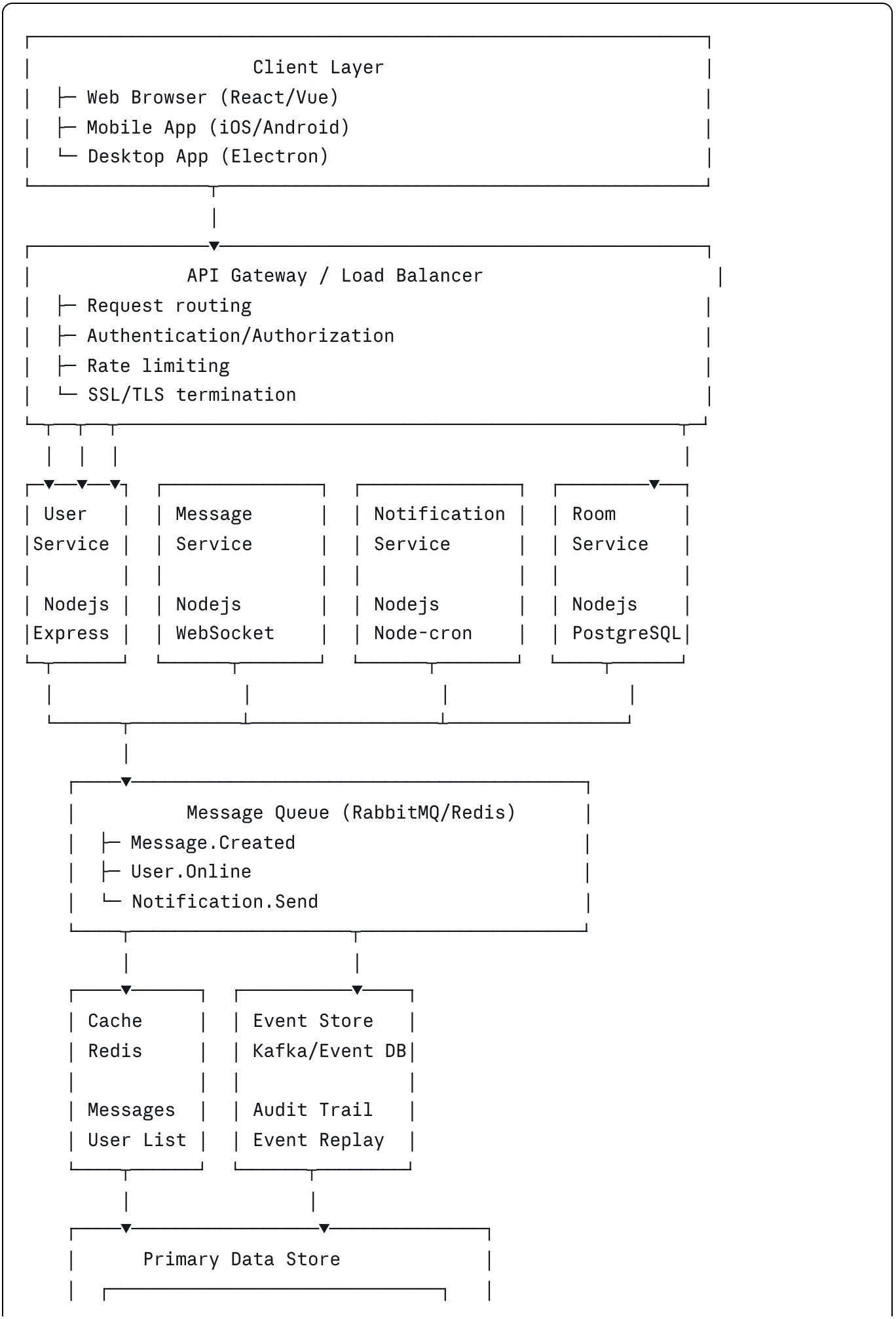
1. "What's the difference between Deployments and StatefulSets?"
2. "How do you handle secrets securely in Kubernetes?"
3. "What's a DaemonSet and when would you use it?"

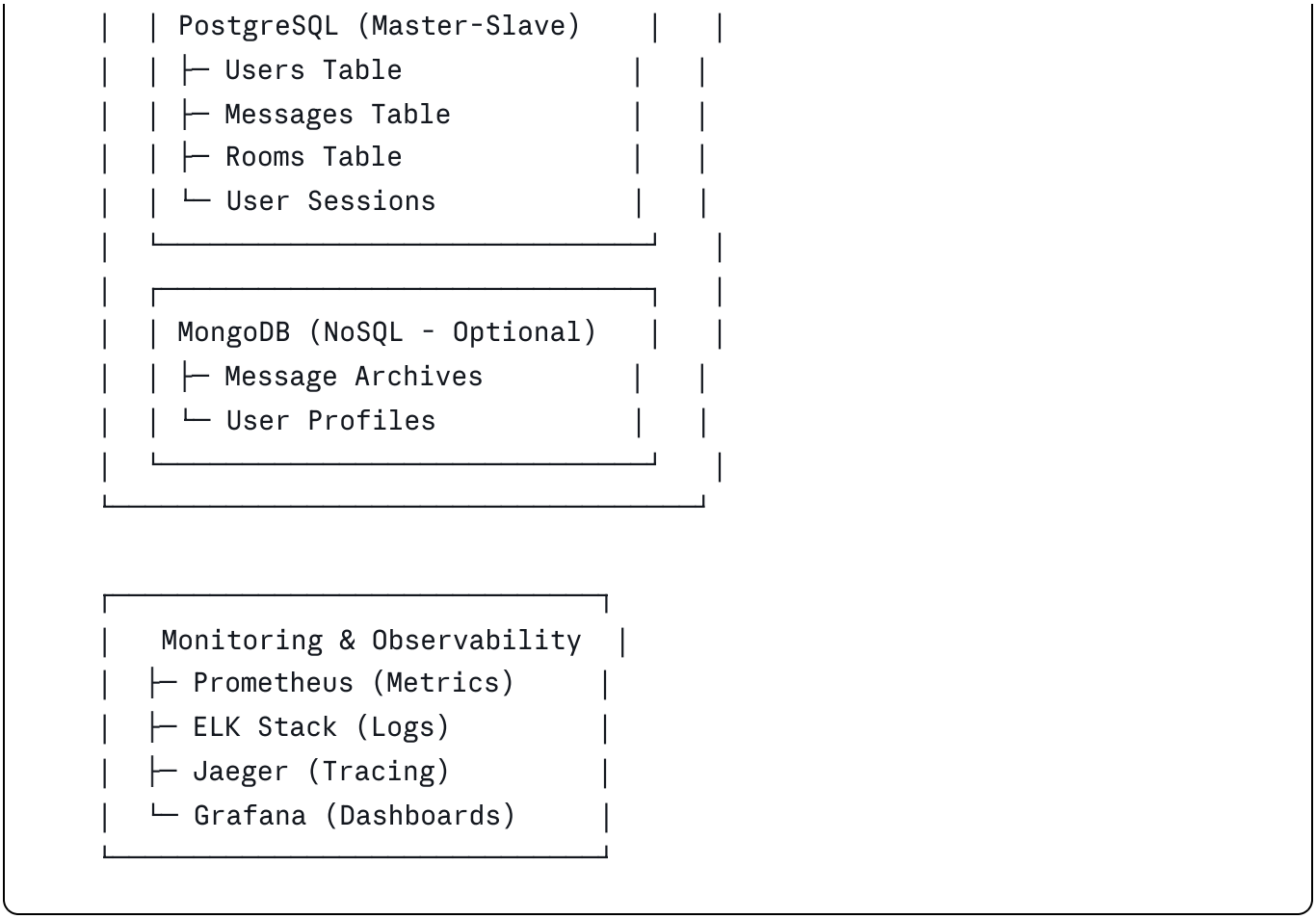
System Design & Architecture

Q20: Design a Scalable Microservices Architecture

Question: "Design a scalable microservices architecture for a real-time chat application. Consider scalability, fault tolerance, and data consistency."

Model Answer:


Client Layer
├─ Web Browser (React/Vue)
├─ Mobile App (iOS/Android)
└─ Desktop App (Electron)
API Gateway / Load Balancer
├─ Request routing
├─ Authentication/Authorization
├─ Rate limiting
└─ SSL/TLS termination
User Service
Nodejs Express
Message Service
Nodejs WebSocket
Notification Service
Nodejs Node-cron
Room Service
Nodejs PostgreSQL
Message Queue (RabbitMQ/Redis)
├─ Message.Created
├─ User.Online
└─ Notification.Send
Cache
Redis
Messages
User List
Event Store
Kafka/Event DB
Audit Trail
Event Replay
Primary Data Store



Technology Stack:

typescript

```
// Microservices
- Language: TypeScript + Node.js
- Framework: Express.js / NestJS
- Real-time: WebSocket (Socket.io) or gRPC

// Message Queue
- RabbitMQ for guaranteed delivery
- Redis for caching + pub/sub

// Databases
- PostgreSQL for ACID transactions
- MongoDB for flexible schema
- Redis for real-time data

// DevOps
- Docker for containerization
- Kubernetes for orchestration
- Helm for deployment management

// Monitoring
- Prometheus for metrics
- ELK for centralized logging
- Jaeger for distributed tracing
```

Service Details:

typescript

```
// 1. User Service
@Controller('users')
export class UserController {
    @Post('register')
    async register(@Body() dto: RegisterDto) {
        // Create user
        // Publish User.Registered event
        // Send welcome email
    }

    @Get('profile')
    async getProfile(@Headers('authorization') token: string) {
        // Validate JWT
        // Return user profile
    }

    @Put('profile')
    async updateProfile(@Body() dto: UpdateProfileDto) {
        // Update profile
        // Publish User.Updated event
    }
}

// 2. Message Service (WebSocket)
@WebSocketGateway()
export class MessageGateway {
    @SubscribeMessage('message')
    async handleMessage(
        @MessageBody() message: CreateMessageDto,
        @ConnectedSocket() socket: Socket
    ) {
        // Save message to DB
        // Publish Message.Created event
        // Broadcast to room subscribers
        // Update message count cache
        this.server.to(message.roomId).emit('message', {
            ...message,
            timestamp: new Date(),
            authorId: socket.userId
        });
    }

    @SubscribeMessage('typing')
```

```
    async handleTyping(
        @MessageBody() data: any,
        @ConnectedSocket() socket: Socket
    ) {
        // Broadcast typing indicator
        socket.to(data.roomId).emit('userTyping', { userId: socket.userId });
    }
}

// 3. Notification Service
@Injectable()
export class NotificationService {
    constructor(
        private readonly emailService: EmailService,
        private readonly smsService: SmsService
    ) {}

    async handleMessageCreated(event: MessageCreatedEvent) {
        // Send notification to room members
        // Respect user preferences
        // Handle offline users
    }

    async handleUserOnline(event: UserOnlineEvent) {
        // Notify friends
        // Update last seen
    }
}

// 4. Room Service
@Controller('rooms')
export class RoomController {
    @Get()
    async listRooms(@Query('skip') skip: number, @Query('take') take: number) {
        // Return paginated rooms
        // Return member count from cache
    }

    @Post()
    @UseGuards(AuthGuard)
    async createRoom(@Body() dto: CreateRoomDto) {
        // Create room
        // Add creator as member
        // Publish Room.Created event
```

```
    }

    @Get(':id/messages')
    async getRoomMessages(@Param('id') roomId: string) {
        // Return paginated messages
        // Use cache for recent messages
    }
}
```

Event-Driven Flow:

typescript

```
// 1. User sends message
POST /messages
Body: { content: "Hello", roomId: "room-123" }
     ↓
// 2. Message Service processes
- Validate message
- Save to PostgreSQL
- Publish "Message.Created" event to RabbitMQ
     ↓
// 3. Multiple Services listen
Message.Created event triggers:
  ├─ Broadcast via WebSocket
  ├─ Update message count cache
  ├─ Publish "Notification.Send" event
  └─ Update Room.lastMessageAt
     ↓
// 4. Notification Service
- Get user notification preferences
- Send notification if enabled
- Handle offline users (store for later)
```

Handling Scalability:

typescript

```
// Horizontal Scaling
- Each service runs in multiple pods (k8s replicas)
- Load balancer distributes requests
- Services communicate asynchronously via RabbitMQ

// Caching
- Redis cache for frequently accessed data
- Cache invalidation on updates
- Cache warming for critical data

// Database Optimization
- Read replicas for scaling reads
- Sharding for scaling writes
- Archiving old messages to cold storage

// Real-time Communication
- WebSocket with multiple backends behind load balancer
- Use message queue for cross-instance messaging
- Sticky sessions for user continuity
```

Fault Tolerance:

typescript

```
// Circuit Breaker
const circuitBreaker = new CircuitBreaker(
    async () => await externalService.call(),
    { timeout: 3000, errorThresholdPercentage: 50 }
);

// Retry with exponential backoff
async function callWithRetry(fn: () => Promise<any>, maxRetries = 3) {
    for (let i = 0; i < maxRetries; i++) {
        try {
            return await fn();
        } catch (error) {
            if (i === maxRetries - 1) throw error;
            await delay(Math.pow(2, i) * 1000);
        }
    }
}

// Graceful shutdown
process.on('SIGTERM', async () => {
    // Stop accepting new connections
    // Wait for existing requests to complete
    // Close database connections
    process.exit(0);
});
```

Behavioral & Soft Skills

Q21: Tell Me About a Time You Had to Debug a Complex Production Issue

Question: "Tell me about a time you had to debug a complex production issue. What was the problem, how did you approach it, and what did you learn?"

Model Answer:

Situation (Context): "At a fintech company, we had a critical issue where payment processing was failing intermittently for about 2% of transactions. Users were experiencing timeouts and their payments weren't being processed, causing customer complaints."

Task (Challenge): "The issue was in production affecting customers, and I needed to diagnose and fix it quickly without breaking anything else."

Action (What I Did):

1. **Immediate Triage (5 minutes)**
    - Checked system health dashboards
    - Reviewed error logs and monitoring alerts
    - Found that API Gateway was returning 504 Gateway Timeout errors
    - Timeouts occurred randomly, not following a pattern
2. **Root Cause Analysis (30 minutes)**
    - Examined application logs (CloudWatch)
    - Found payment service taking 60+ seconds occasionally
    - Checked database query logs - identified a slow SQL query
    - Query: `SELECT * FROM transactions WHERE user_id = ? AND DATE = TODAY()`
    - This query was missing an index and doing full table scans
    - The problem worsened during peak hours
3. **Temporary Fix (Immediate)**

typescript

```typescript
// Added timeout protection
const paymentPromise = processPayment();
const timeoutPromise = new Promise((_, reject) =>
    setTimeout(() => reject(new Error('Timeout')), 5000)
);

return Promise.race([paymentPromise, timeoutPromise]);
```

4. **Permanent Fix (1 hour)**

sql

```sql
-- Added missing index
CREATE INDEX idx_transactions_user_date
ON transactions(user_id, transaction_date)
WHERE status = 'pending';

-- Optimized query
SELECT id, amount, status
FROM transactions
WHERE user_id = ?
AND transaction_date = CURRENT_DATE
LIMIT 100;
```

5. **Deployment**
    - Deployed fix during off-peak hours
    - Monitored metrics closely
    - Error rate dropped to <0.01%

Result: "The issue was resolved in 2 hours. Payment processing returned to normal, zero data loss, and no customer refunds needed."

What I Learned:

1. **Index optimization** - Always include indexes for commonly filtered columns
2. **Monitoring** - The alerts could have been better (by time taken, not just HTTP status)
3. **Load testing** - We should have tested at realistic data volumes before production
4. **Communication** - Kept stakeholders updated every 15 minutes during incident

Follow-up Actions:

- Added APM (Application Performance Monitoring) using DataDog
- Implemented database query monitoring and slow query alerts
- Created runbook for similar issues
- Conducted post-mortem with team

Q22: Tell Me About Your Experience with Microservices and System Design

Question: "Tell me about a system you've designed or worked on that required microservices. What challenges did you face and how did you solve them?"

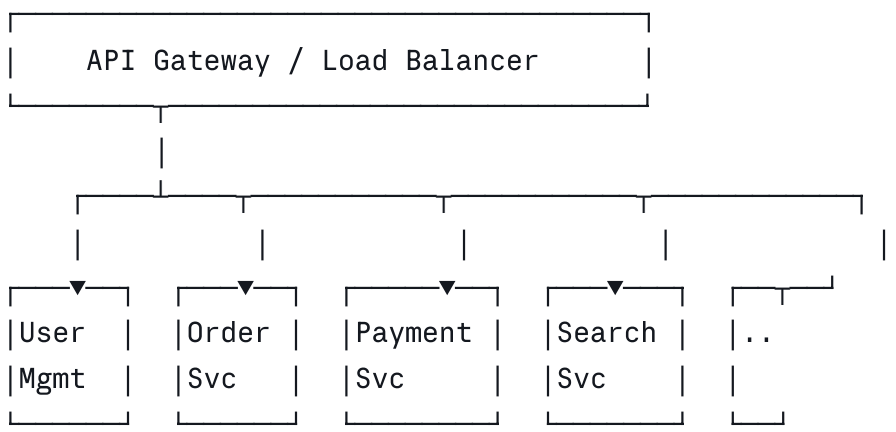
Model Answer:

Project: E-Commerce Platform Redesign

Situation: "At an e-commerce company, we had a monolithic Ruby on Rails application that was becoming difficult to scale. We were experiencing:

- 30-minute deployments
- Database locks during peak traffic
- Team dependencies (10 developers, 1 codebase)
- Hard to scale specific services (payment vs. search)"

Solution:




```
Message Queue: RabbitMQ
- Order.Created
- Payment.Processed
- Inventory.Updated
- Notification.Send
```

Challenges & Solutions:

1. **Data Consistency**
    - Challenge: Distributed transactions across services
    - Solution: Event sourcing + CQRS pattern

typescript

```
// Order Service publishes event
await publishEvent({
  type: 'OrderCreated',
  orderId: 'ORD-123',
  userId: 'USR-456',
  items: [...]
});

// Payment Service subscribes
onOrderCreated: async (event) => {
  // Process payment
  // Publish PaymentCompleted event
}

// Inventory Service subscribes
onPaymentCompleted: async (event) => {
  // Deduct inventory
  // Handle backorder if needed
}
```

2. **Service Communication**
   - Challenge: Sync vs Async communication
   - Solution: Hybrid approach

typescript

```
// Sync for immediate responses
POST /orders
├─ Create order (immediate)
├─ Validate payment method (immediate)
└─ Return order ID (immediate)

// Async for non-blocking operations
- Process payment (async via queue)
- Update inventory (async)
- Send confirmation email (async)
```

3. **Deployment & DevOps**
   - Before: 30-minute monolith deployments
   - After: 2-minute individual service deploys

```yaml
# Each service has own pipeline
- Build Docker image
- Run tests
- Push to registry
- Deploy via Helm
```

4. **Database Per Service Pattern**
   - Challenge: Maintaining referential integrity
   - Solution: Service ownership + eventual consistency

```typescript
// Order Service owns orders table
// Payment Service owns payments table
// Search Service has read-only denormalized data

// Data synchronization via events
OrderCreated → Search Service updates Elasticsearch
```

Results:

- Deployment time: 30min → 2min
- Scalability: Could scale payment service 10x independently
- Development velocity: 10 developers could work in parallel
- Uptime: Improved from 99.5% → 99.95%

Lessons Learned:

1. Microservices add complexity - start monolithic
2. Event sourcing is powerful but needs investment
3. Monitoring becomes critical with distributed systems
4. Data consistency is harder than it seems

Thank you! Your resume and experience make you a strong candidate. Any final questions?